Projet – MLSF : Un solveur SAT pour la résolution du jeu de réflexion Meowdoku

Partie II – Un langage de contraintes compilé vers SAT

Marc Aiguier - Pascale Le Gall - Romain Pascual

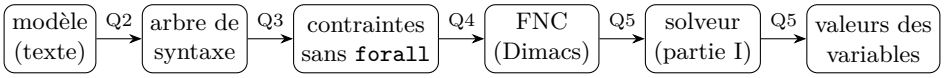
26 octobre 2026

Avant-propos

Les éléments de la partie I sur le travail en binôme, l'usage des assistants IA et les critères d'évaluation restent valables pour la partie II. Le solveur DPLL de la partie I est réutilisé tel quel : c'est lui qui résout les formules produites dans cette partie.

La date limite de dépôt de la partie II est le **16 novembre 2026**.

Dans la partie I, la formule d'une grille de Meowdoku était produite par un programme écrit spécifiquement pour ce jeu. Chaque nouveau problème demanderait un nouveau programme de ce type. L'objectif de la partie II est d'écrire, une fois pour toutes, un *compilateur* : il lit la description d'un problème dans un petit langage de contraintes, la traduit en forme normale conjonctive (FNC), appelle votre solveur, puis traduit le modèle trouvé dans les termes du problème de départ.



Ce compilateur mobilise l'ensemble du cours : la logique propositionnelle et la mise en forme normale, les grammaires algébriques, l'analyse lexicale et l'analyse syntaxique. La question 6, enfin, porte sur la *modélisation* : on y verra que, pour un même problème, le choix du modèle peut compter bien davantage que les performances du solveur.

Le langage de programmation préconisé reste Python. L'analyse syntaxique utilise la bibliothèque Lark (`https://github.com/lark-parser/lark`, `pip install lark`).

Deux dossiers accompagnent cet énoncé :

- `proplog/`, le code de départ, qui traite le fragment propositionnel du langage ;
- `exemples/`, quatre modèles écrits dans le langage complet : `pigeonhole.csp`, `queens.csp`, `coloring.csp` et `sudoku.csp`.

Recommandation générale : lisez l'énoncé jusqu'au bout avant de commencer. La question 1 construit une première chaîne complète, du texte au solveur, pour le seul fragment propositionnel ; les questions 2 à 5 en enrichissent chacune une étape. Avancez-les ensemble sur un petit fragment du langage (par exemple les variables entières et les comparaisons, sans `forall`), puis élargissez. Comme pour la partie I, il vaut mieux un compilateur plus petit, bien testé et bien compris, qu'un compilateur complet que vous ne savez pas expliquer.

Ce que l'on veut pouvoir écrire

Le principe des tiroirs dit que l'on ne peut pas ranger trois pigeons dans deux trous sans en mettre deux dans le même trou. Dans le fragment propositionnel (fichier `pigeonhole.csp` du dossier `proplog/examples/`), il faut l'écrire à la main : une variable booléenne `pij` par couple

(pigeon i, trou j), une contrainte par pigeon, une contrainte par trou. La formule correspondante est

$$\bigwedge_{i=1}^{3}(p_{i1} \vee p_{i2}) \ \wedge \ \bigwedge_{j=1}^{2} \ \bigwedge_{1 \leq i < i' \leq 3} \neg(p_{ij} \wedge p_{i'j}),$$

où la première conjonction dit que chaque pigeon est dans un trou, et la seconde qu'aucun trou ne contient deux pigeons. Dans le fragment propositionnel, elle s'écrit :

```
var p11 : bool;  var p12 : bool;
var p21 : bool;  var p22 : bool;
var p31 : bool;  var p32 : bool;

// each pigeon is in a hole
constraint p11 or p12;
constraint p21 or p22;
constraint p31 or p32;

// no hole holds two pigeons
constraint not (p11 and p21) and not (p11 and p31) and not (p21 and p31);
constraint not (p12 and p22) and not (p12 and p32) and not (p22 and p32);
```

Avec un pigeon et un trou de plus, il faudrait déjà 12 variables et réécrire toutes les contraintes. Dans le langage de cette partie, le même problème, pour quatre pigeons et trois trous, s'écrit (`exemples/pigeonhole.csp`) :

```
// Pigeonhole: 4 pigeons, 3 holes, each pigeon in its own hole: UNSAT.

const P = 4;
const H = 3;

array hole[P] : 1..H;

constraint allDifferent(hole[*]);
```

La case `hole[p]` du tableau `hole` est le numéro du trou du pigeon `p`, entre 1 et `H` ; la contrainte `allDifferent` impose que deux pigeons ne soient jamais dans le même trou. Changer la taille du problème revient à changer `P` et `H`. Ce sont les variables booléennes et les contraintes du premier fichier que le compilateur doit produire à partir du second.

Les trois autres modèles du dossier `exemples/` montrent ce que le langage permet d'exprimer. Le problème des huit reines place huit reines sur un échiquier sans que deux d'entre elles soient sur une même ligne, une même colonne ou une même diagonale (`exemples/queens.csp`) :

```
// N queens: queen[i] is the column of the queen of row i.

const N = 8;

array queen[N] : 1..N;

// one queen per column
constraint allDifferent(queen[*]);

// no two queens on a diagonal: arithmetic is only allowed on known
// values, so the diagonals are forbidden value by value
forall i in 1..N, j in 1..N, a in 1..N, b in 1..N {
    constraint i < j and abs(a - b) = j - i -> not (queen[i] = a and queen[j] = b);
}
```

La coloration d'un graphe donne une couleur à chaque sommet, de sorte que deux sommets voisins n'aient jamais la même couleur. Les arêtes du graphe sont des données, rangées dans un tableau constant (`exemples/coloring.csp`) :

```
// Graph colouring with 3 colours: the Petersen graph (10 vertices, 15 edges,
// chromatic number 3).

enum Color = { Red, Green, Blue };
```

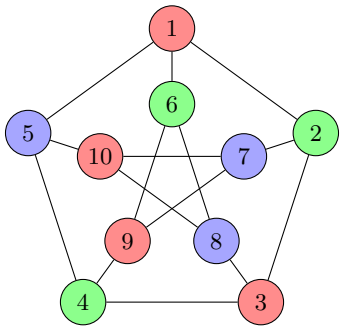
```
const V = 10;
const E = 15;
const edge[E][2] = [
    [1, 2], [2, 3], [3, 4], [4, 5], [5, 1],     // outer cycle
    [6, 8], [8, 10], [10, 7], [7, 9], [9, 6],   // inner star
    [1, 6], [2, 7], [3, 8], [4, 9], [5, 10]     // spokes
];

array color[V] : Color;

forall e in 1..E {
    constraint color[edge[e][1]] != color[edge[e][2]];
}

// symmetry breaking: the colours of vertex 1 and 2 are fixed
constraint color[1] = Red;
constraint color[2] = Green;
```

Le graphe de Petersen, avec une coloration qui satisfait ce modèle (rouge, vert, bleu) :



Le dernier, `exemples/sudoku.csp`, décrit les règles du Sudoku et une grille à compléter.

La syntaxe de ces modèles est décrite plus bas. L'objectif de la partie II est que votre compilateur accepte ces quatre modèles et les résolve à l'aide de votre solveur de la partie I. Ils servent aussi de premiers tests (tableau 1).

fichier	problème	réponse attendue
`pigeonhole.csp`	4 pigeons, 3 trous	insatisfiable
`queens.csp`	8 reines	satisfiable
`coloring.csp`	graphe de Petersen, 3 couleurs	satisfiable
`sudoku.csp`	une grille de Sudoku	satisfiable

TABLE 1 – Les modèles du dossier `exemples/` et les réponses attendues.

Point de départ : le fragment propositionnel

Le dossier `proplog/` traite le fragment propositionnel du langage : des variables booléennes et des contraintes écrites avec `not`, `and`, `or`, `->` et des parenthèses.

```
var x : bool;
var y : bool;
var z : bool;

constraint x or y;
constraint (not x) and z;
constraint z -> y;
```

Les connecteurs sont, par priorité décroissante, `not`, `and`, `or` et `->` ; `and` et `or` sont associatifs à gauche, `->` est associatif à droite. Le dossier contient :

- `syntax.py` : la grammaire Lark du fragment (`GRAMMAR`, analyseur LALR), une classe Python par nœud de l'arbre de syntaxe abstraite (`Program`, `VarDecl`, `Constraint`, `Var`, `Not`, `And`,

`Or`, `Implies`), la classe `ASTBuilder` qui construit cet arbre à partir de l'arbre d'analyse de Lark, la fonction `parse` (du texte à l'arbre) et la fonction `pretty`, qui affiche l'arbre en parenthésant chaque connecteur ;

- `csp.py` : la ligne de commande `python csp.py modele.csp`, qui affiche l'arbre du modèle ;
- `examples/` : quatre modèles (l'exemple ci-dessus, les priorités et associativités, le principe des tiroirs, une erreur de syntaxe) ;
- `tests/test_syntax.py` : des tests de l'analyseur, à lancer depuis le dossier `proplog/` par la commande `python -m tests.test_syntax` ;
- `README.md` : une présentation du fragment et de l'arbre construit pour chaque construction.

Ce code s'arrête à l'arbre de syntaxe : vérifier les noms (`constraint w;` doit être rejeté si `w` n'est pas déclarée), évaluer les formules, produire la formule sous FNC et appeler le solveur sont les premières étapes du projet (Question 1). De plus, la grammaire et l'arbre ont été écrits pour être étendus : par exemple, le domaine `bool` est déjà représenté comme un domaine énuméré particulier, `EnumDomain("bool")`.

Ce code peut être étendu, modifié ou réécrit entièrement. Il sert surtout de prise en main du projet.

Le langage à compiler

Le langage à compiler ajoute au fragment propositionnel des variables à domaine fini, des tableaux, des boucles `forall` et des contraintes globales. Le modèle suivant en montre l'essentiel.

```
// les commentaires commencent par //
const N = 9;                            // constante entière
enum Color = { Red, Green, Blue };      // type énuméré
const region[2][3] = [[1, 1, 2],        // tableau constant (données)
                      [3, 3, 2]];

var b : bool;                           // variables : booléenne,
var x : 1..N;                           //   entière (intervalle),
var c : Color;                          //   énumérée
array grid[N][N] : 1..N;                // tableau, indices à partir de 1
array used[Color] : bool;               // indicé par un type énuméré

constraint x != 3 and (b -> c = Red);   // expression booléenne
constraint allDifferent(grid[1][*]);    // contrainte globale, tranche

forall i in 1..N, j in i..N {           // boucle, déroulée à la compilation
    constraint i < N -> grid[i][j] != grid[i + 1][j];
}
forall c2 in Color { constraint used[c2]; }   // boucle sur un type énuméré
```

Mots-clés. Les onze mots suivants sont réservés : ils ne peuvent pas servir de noms.

mot-clé	rôle
`const`	constante (entier ou tableau de données)
`enum`	type énuméré
`var`	variable
`array`	tableau de variables
`bool`	domaine booléen
`constraint`	contrainte
`forall`, `in`	boucle `forall i in 1..N { ... }`
`not`, `and`, `or`	connecteurs

Six d'entre eux, `var`, `bool`, `constraint`, `not`, `and` et `or`, sont déjà réservés dans `proplog/`. Les noms des contraintes globales (`allDifferent`, `atMostOne`, etc.) et `abs` ne sont pas des mots-clés, mais des noms de fonctions. Les autres lexèmes sont les noms (lettres, chiffres et `_`, sans chiffre en tête), les entiers, les commentaires (de `//` à la fin de la ligne) et les symboles `; : , .. [ ] { } ( ) -> = != < <= > >= + - *` ; `*` est à la fois la multiplication et le joker des tranches.

Déclarations. Une constante est un entier (`const`) ou un tableau d'entiers ou de valeurs énumérées. Un type énuméré (`enum`) déclare des noms de valeurs, ordonnées dans l'ordre de déclaration. Une variable (`var`) a pour domaine `bool`, un intervalle d'entiers `a..b` ou un type énuméré ; un tableau de variables (`array`) a une ou plusieurs dimensions, chacune donnée par une taille n (indices $1..n$) ou par un type énuméré. Tous les noms (constantes, types, valeurs énumérées, variables, tableaux, indices de boucle) partagent un même espace de noms.

Expressions.

- Connecteurs du fragment propositionnel, avec les mêmes priorités.
- Comparaisons `=`, `!=`, `<`, `<=`, `>`, `>=`, entre deux entiers ou deux valeurs d'un même type énuméré ; seules `=` et `!=` s'appliquent aux booléens. Elles sont plus prioritaires que `not` (`not x = 2` se lit `not (x = 2)`) et ne s'enchaînent pas (`a = b = c` est une erreur de syntaxe).
- Arithmétique `+`, `-`, `*`, moins unaire et `abs(e)`, avec les priorités usuelles, *uniquement sur des valeurs connues* (voir ci-dessous).
- Accès aux tableaux `grid[i][j]`, et tranches `grid[i][*]`, `grid[*][*]` qui désignent une liste de cases (ordre ligne par ligne). Une tranche n'est permise qu'en argument d'une contrainte globale.
- Contraintes globales : sur des booléens, `atLeastOne`, `atMostOne` et `exactlyOne` ; sur des entiers ou des valeurs énumérées, `allDifferent`. Leurs arguments mélangent librement expressions et tranches. Syntaxiquement, ce sont de simples appels de fonction : ajouter une contrainte globale ne modifie pas la grammaire.

Valeurs connues et déroulement. Une valeur est *connue* si elle est fixée à la compilation : littéraux, constantes, valeurs énumérées, indices de boucle, cases d'un tableau constant, et tout ce qui se calcule à partir d'eux seulement. Les tailles de tableaux, les bornes de domaines, les indices de tableau et les opérandes de l'arithmétique doivent être connus. Ainsi `grid[i + 1][j]` est correct dans une boucle sur `i`, mais `x + 1 = y` est une erreur si `x` est une variable. Chaque `forall` peut alors être remplacé par la conjonction de ses instances, et les parties connues des contraintes être calculées.

Ce calcul est *paresseux* (court-circuit) : dans

```
forall i in 1..N { constraint i < N -> grid[i + 1][1] != grid[i][1]; }
```

la condition `i < N` est fausse pour `i = N` : l'implication est vraie, la contrainte disparaît, et l'accès `grid[N + 1][1]`, hors du tableau, n'est jamais évalué. Une condition sur des valeurs connues joue donc le rôle d'un filtre. En revanche, un accès hors du tableau qui est effectivement évalué est une erreur de compilation.

Erreurs. Un modèle erroné (syntaxe, nom non déclaré, type, valeur inconnue là où une valeur connue est attendue, indice hors bornes, mauvais nombre d'arguments) doit être rejeté avec un message qui explique l'erreur, plutôt que de produire une formule fausse ou de faire échouer le compilateur sur une exception Python.

Interface conseillée. Cette interface n'est pas imposée, mais elle permettra aux enseignants d'exécuter plus simplement vos programmes lors de l'évaluation. Si vous en adoptez une autre, veuillez l'indiquer dans votre rendu, avec les commandes à lancer et le format de la sortie.

```
python csp.py model.csp                  # FNC au format Dimacs sur la sortie standard
python csp.py model.csp --dimacs f.cnf   # FNC écrite dans f.cnf
python csp.py model.csp --solve          # résolution par votre solveur
```

Avec `--solve`, la sortie indique `s SATISFIABLE` ou `s UNSATISFIABLE` (comme pour la partie I), puis, si la formule est satisfiable, une ligne `nom = valeur` par variable, par exemple `grid[1][2] = 5` ou `color[3] = Red`.

Vous pouvez enrichir le langage, à condition qu'il accepte toujours les modèles écrits dans le langage ci-dessus.

Question 1 – Du fragment propositionnel à votre solveur

Le code de départ produit l'arbre de syntaxe d'un modèle du fragment propositionnel. Il s'agit de compléter la chaîne jusqu'au solveur de la partie I : à partir de cet arbre, produire une FNC,

l'écrire au format Dimacs, appeler votre solveur, puis afficher le modèle trouvé dans les termes du modèle de départ (`x = true`, par exemple). Quelques pistes :

- **Table des symboles.** Le code de départ accepte une contrainte sur une variable non déclarée : une première table des symboles permet de rejeter ce modèle. Elle sera étendue à la question 3.
- **Mise en FNC.** La transformation de Tseitin [Tse68] (vue en TD 2) introduit une variable auxiliaire par sous-formule. On peut l'économiser lorsqu'une contrainte est déjà une clause, ou s'y ramène directement (`a -> b`, `not (a and b)`).
- **Noms des variables.** Le format Dimacs ne connaît que des numéros de variables : il faut garder la correspondance entre noms et numéros, par exemple dans des lignes de commentaire du fichier (`c var 3 z`), pour pouvoir traduire le modèle renvoyé par le solveur. Les variables auxiliaires n'ont pas de nom et ne sont pas affichées.
- **Validation.** Une fonction qui évalue une formule pour une valuation donnée permet de tester la chaîne sur des formules engendrées aléatoirement avec peu de variables : la FNC doit être satisfiable si et seulement si la formule l'est, et la restriction d'un modèle de la FNC aux variables nommées doit être un modèle de la formule. Ces tests serviront de filet de sécurité pour toute la suite.

Question 2 – Analyse syntaxique

Il est demandé d'étendre la grammaire Lark du fragment propositionnel au langage complet, et de construire l'arbre de syntaxe abstraite des modèles.

Grammaire. Étendre la grammaire au langage complet, en restant en LALR(1) (utiliser l'option `parser="lalr"`). Les priorités et associativités doivent être encodées par la grammaire elle-même, par niveaux successifs, comme dans la grammaire du fragment. Si Lark signale un conflit pendant votre travail, cherchez à le comprendre avant de le lever. Les mots-clés de la description du langage doivent rester réservés.

Arbre de syntaxe abstraite. Construire un arbre de syntaxe abstraite des modèles.Par exemple, on pourra ajouter une classe Python par construction du langage et étendre `ASTBuilder`. Pour vous aider, vous pouvez étendre le système d'impression élégante (*pretty print* en anglais) pour afficher l'arbre sous forme de texte, et vérifier sur des exemples que l'analyse du texte affiché redonne le même arbre. Vous pouvez aussi étendre les tests : priorités des nouveaux opérateurs, erreurs de syntaxe, etc.

A la fin de cette étape les modèles du dossier `exemples/` doivent tous être acceptés.

Question 3 – Analyse et transformation de l'arbre

L'arbre produit à la question 2 reflète fidèlement le texte du modèle, mais il se prête mal à la traduction en FNC : il contient des boucles, des tranches, des noms à résoudre, des expressions calculables dès la compilation. L'objectif de cette question est de parcourir l'arbre pour l'*analyser* et le *transformer*, de sorte que la traduction de la question 4 n'ait plus à traiter que des cas simples. Le découpage en passes, leur ordre et la forme du résultat sont libres. Les pistes ci-dessous précisent ce qui est attendu, mais vous pouvez en laisser de côté, en ajouter, ou procéder autrement, à condition de savoir justifier vos choix.

Analyse. Un modèle syntaxiquement correct peut néanmoins être dénué de sens. Une table des symboles (à quoi correspond chaque nom ?) permet de détecter, avant toute traduction, les erreurs listées dans la description du langage : nom non déclaré ou déclaré deux fois, types incompatibles (comparer une couleur à un entier, additionner des booléens), valeur inconnue là où une valeur connue est exigée (`x + 1 = y`), mauvais nombre d'indices dans un accès à un tableau, indice hors bornes, contrainte globale appelée avec de mauvais arguments, etc.

Transformations et simplifications. Le résultat visé est une liste de contraintes *sans boucle*, qui ne portent plus que sur des cases de variables, par exemple `grid[1][2] != grid[2][2]`. Pour y parvenir, on peut notamment :

- remplacer chaque `forall` par ses instances ;
- calculer les valeurs connues : arithmétique, accès aux tableaux constants, comparaisons entre valeurs connues ;
- simplifier les connecteurs dont un opérande est connu (`faux and e` donne `faux` sans évaluer `e`, etc.), en respectant le court-circuit décrit plus haut ;
- remplacer les tranches par la liste des cases qu'elles désignent ;
- normaliser les expressions pour réduire le nombre de cas à traduire, par exemple réécrire `not (x = 2)` en `x != 2` ou `x > y` en `y < x`.

Une contrainte qui se simplifie en `vrai` disparaît ; une contrainte qui se simplifie en `faux` rend le modèle insatisfiable, ce qui mérite au moins un avertissement.

Tests. Quelques idées de tests : une collection de modèles erronés ; de petits modèles dont on connaît le résultat déroulé et simplifié ; des modèles où un accès hors bornes est protégé par une condition fausse (aucune erreur ne doit être signalée) et d'autres où il ne l'est pas (l'erreur doit être signalée).

Pour aller plus loin. Conserver dans chaque nœud de l'arbre sa position (ligne, colonne) dans le texte, et indiquer cette position dans chaque message d'erreur, par exemple :

```
model.csp: line 3, column 31: index 1 of 'a' is 4, out of range 1..3
```

Pour plus d'information, regarder l'utilisation des `meta` informations dans la documentation de Lark.

Question 4 – Traduction en FNC

Après la question 3, les contraintes ne portent plus que sur des variables, mais celles-ci ne sont pas toutes booléennes : une variable de domaine `1..9` ne peut pas être donnée telle quelle à un solveur SAT. Il faut d'abord représenter chaque variable à domaine fini par des variables booléennes, puis exprimer les comparaisons et les contraintes globales sur ces variables booléennes.

Encodage direct des domaines. Une variable `x` de domaine $\{v_1, \dots, v_k\}$ est représentée par k variables booléennes, notées $x_{v_1}, \dots, x_{v_k}$: x_v est vraie si et seulement si `x` vaut v. Toute valuation de ces k variables ne représente pas une valeur de `x` : il faut ajouter des clauses qui imposent qu'*exactement une* d'entre elles soit vraie, à savoir

- au moins une : la clause $x_{v_1} \vee x_{v_2} \vee \cdots \vee x_{v_k}$;
- au plus une : pour chaque couple $i < j$, la clause $\neg x_{v_i} \vee \neg x_{v_j}$.

Par exemple, pour `var x : 1..3;`, on introduit x_1, x_2, x_3 et les quatre clauses $(x_1 \vee x_2 \vee x_3)$, $(\neg x_1 \vee \neg x_2)$, $(\neg x_1 \vee \neg x_3)$, $(\neg x_2 \vee \neg x_3)$. Une valuation qui satisfait ces clauses rend vraie exactement une des trois variables, et donne donc une valeur à `x`. Une variable `bool` reste une variable booléenne ; les valeurs d'un type énuméré sont numérotées $0, 1, \dots$ dans l'ordre de déclaration et traitées comme des entiers. La première étape est donc de mettre en œuvre l'encodage direct des variables du modèle.

Comparaisons. Une contrainte sur `x` devient alors une formule sur les variables x_v. Avec `x` et `y` de domaine 1..3 :

- `x = 2` devient x_2, et `x != 2` devient $\neg x_2$;
- `x <= 2` peut s'écrire $x_1 \vee x_2$ (`x` prend une des bonnes valeurs) ou $\neg x_3$ (`x` ne prend aucune des mauvaises valeurs) ;
- `x != y` peut s'écrire $\neg(x_1 \wedge y_1) \wedge \neg(x_2 \wedge y_2) \wedge \neg(x_3 \wedge y_3)$, soit trois clauses binaires.

Traduire chaque comparaison en une formule sur les variables x_v. Plusieurs traductions sont souvent possibles, comme le montrent les exemples ci-dessus ; explorez-les (taille de la formule, longueur des clauses, cas d'une comparaison entre deux variables comme `x < y`) et justifiez vos choix. Une fois les comparaisons traduites, il ne reste qu'une formule propositionnelle, que la mise en FNC de la question 1 achève de traduire.

Contraintes globales. Une contrainte globale porte sur un nombre n connu d'expressions (éventuellement après avoir déroulé les tranches). Pour des expressions booléennes $l_1, \ldots, l_n$:

- `atLeastOne`$(l_1, \ldots, l_n)$ est vraie lorsqu'au moins une des l_i est vraie ;
- `atMostOne`$(l_1, \ldots, l_n)$ est vraie lorsqu'au plus une des l_i est vraie ;
- `exactlyOne`$(l_1, \ldots, l_n)$ est vraie lorsqu'exactement une des l_i est vraie.

Pour des expressions entières ou énumérées $e_1, \ldots, e_n$:

- `allDifferent`$(e_1, \ldots, e_n)$ est vraie lorsque les valeurs des e_i sont deux à deux distinctes.

Les clauses de l'encodage direct précédent expriment exactement `exactlyOne`$(x_{v_1}, \ldots, x_{v_k})$.

Traduire `atLeastOne`, `atMostOne`, `exactlyOne` et `allDifferent` (pour `allDifferent`, on pourra remarquer que, pour chaque valeur v, au plus une des variables vaut v). Pour `atMostOne` parmi n littéraux, on utilisera l'encodage *deux à deux*, comme ci-dessus pour les domaines : une clause binaire par couple de littéraux, soit $n(n-1)/2$ clauses et aucune variable auxiliaire.

Pour aller plus loin. Le nombre de clauses de l'encodage deux à deux pour `atMostOne` croît comme n^2. Le *compteur séquentiel* de Sinz [Sin05] n'en utilise qu'un nombre linéaire, au prix de variables auxiliaires. Pour `atMostOne`$(x_1, \ldots, x_n)$, on introduit $n-1$ variables $s_1, \ldots, s_{n-1}$, où s_i signifie "l'un au moins de $x_1, \ldots, x_i$ est vrai", et les clauses

$$\neg x_1 \vee s_1, \qquad \left.\begin{array}{l} \neg x_i \vee s_i \\ \neg s_{i-1} \vee s_i \\ \neg x_i \vee \neg s_{i-1} \end{array}\right\} \text{ pour } 1 < i < n, \qquad \neg x_n \vee \neg s_{n-1},$$

soit $3n-4$ clauses. Les deux premières familles propagent l'information "un x_j vrai a déjà été rencontré" le long de la suite ; la troisième interdit alors tout nouveau x_i vrai. Implémenter cet encodage et le comparer à l'encodage deux à deux.

Attention : les clauses n'expriment "au plus un" que pour une valeur bien choisie des s_i ; peut-on encore les utiliser sous une négation, par exemple dans `b -> not atMostOne(...)` ?

Question 5 – Jonction avec la partie I

Cette question raccorde le compilateur au solveur de la partie I, comme la question 1 l'a fait pour le fragment propositionnel. Il s'agit d'écrire la FNC produite par la question 4 au format Dimacs, d'appeler votre solveur, puis, à partir du modèle qu'il renvoie, de retrouver la valeur de chaque variable du modèle de départ (celle des v tels que x_v est vraie) et de l'afficher dans les termes du modèle, par exemple `grid[1][2] = 5`.

Vérifier sur les modèles du dossier `exemples/` que les réponses sont celles du tableau 1, et que les solutions obtenues sont correctes. Cette vérification peut être faite manuellement, vous pouvez aussi écrire un vérificateur, par exemple, un programme qui vérifie qu'une grille de Sudoku remplie respecte les règles et les cases données. Vous pouvez aussi ajouter vos propres modèles et les vérifier.

Question 6 – Modéliser : le modèle compte plus que le solveur

Les grilles de Meowdoku sont décrites dans le format de la partie I. On en tire les données d'un modèle :

```
const N = 6;      // lignes
const M = 6;      // colonnes
const K = 6;      // régions
enum Zone = { A, B, C, D, E, F };
const region[N][M] = [
    [A, B, C, B, B, D],
    [A, B, B, B, B, D],
    [A, A, B, D, D, D],
    [A, A, A, E, D, D],
    [A, A, A, A, D, D],
    [F, A, D, D, D, D]
];
```

Un modèle se compose alors des données de la grille, qui changent d'une grille à l'autre, suivies des règles du jeu (partie I, question 1), écrites une fois pour toutes. Cette question compare deux façons d'écrire ces règles, d'abord pour des grilles où $N = M = K$.

Premier modèle La position du chat de chaque région.

```
array row[Zone] : 1..N;
array col[Zone] : 1..M;

// au plus un chat par ligne et par colonne
constraint allDifferent(row[*]);
constraint allDifferent(col[*]);

// le chat de la région z est dans la région z
forall z in Zone, i in 1..N, j in 1..M {
    constraint region[i][j] != z -> not (row[z] = i and col[z] = j);
}

// pas deux chats voisins (en diagonale : lignes et colonnes sont distinctes)
forall z1 in Zone, z2 in Zone, i in 1..N, j in 1..M,
       k in (i - 1)..(i + 1), l in (j - 1)..(j + 1) {
    constraint z1 < z2 and k != i and l != j
               -> not (row[z1] = i and col[z1] = j and row[z2] = k and col[z2] = l);
}
```

Le tableau `row` est indicé par le type `Zone` : il a une case par région, `row[A]`, ..., `row[F]`. La valeur de `row[z]`, entre 1 et N, est le numéro de la ligne où se trouve le chat de la région `z` ; de même, `col[z]`, entre 1 et M, est le numéro de sa colonne. Le couple `(row[z], col[z])` est donc la position du chat de la région `z` : ce modèle décrit une solution par la liste des positions des chats, région par région.

Second modèle Une variable booléenne par case. Comme dans la partie I, une variable `cat[i][j]` indique si la case (i, j) porte un chat.

```
array cat[N][M] : bool;

// exactement un chat par ligne et par colonne (N = M = K)
forall i in 1..N { constraint exactlyOne(cat[i][*]); }
forall j in 1..M { constraint exactlyOne(cat[*][j]); }

// pas deux chats voisins (en diagonale : lignes et colonnes sont exclusives)
forall i in 1..(N - 1), j in 1..M, l in (j - 1)..(j + 1) {
    constraint l >= 1 and l <= M and l != j -> not (cat[i][j] and cat[i + 1][l]);
}

// au plus un chat par région : à compléter
```

6.1 Écrire un programme qui convertit un fichier de grille de la partie I en les déclarations ci-dessus.

6.2 Compléter le second modèle par la règle "au plus un chat par région". Résoudre des grilles avec chacun des deux modèles, et vérifier les solutions trouvées à l'aide d'un vérificateur indépendant.

6.3 Comparer les deux modèles sur des grilles de tailles croissantes (6, 8, 10, ...), à l'aide des mesures de la partie I (question 5). Le compilateur et le solveur sont les mêmes : comment expliquer les écarts observés ?

6.4 Contraintes redondantes. Ajouter au second modèle la règle "au moins un chat par région", pourtant conséquence des autres quand $N = M = K$. Aucune tranche ne désigne les cases d'une région : trouver une façon de l'écrire dans le langage (ou étendre le langage, en le justifiant). Comparer les temps de résolution avec et sans cette règle sur des grilles de plus en plus grandes, et expliquer.

6.5 Généraliser le second modèle aux grilles quelconques (N, M et K libres), et vérifier les solutions trouvées par un vérificateur indépendant. Sur de petites grilles, comparer les réponses SAT/UNSAT avec une recherche exhaustive.

6.6 Conclure : dans vos expériences, qu'est-ce qui a le plus d'effet sur le temps de résolution, les heuristiques et optimisations du solveur (partie I) ou le choix du modèle ?

Questions optionnelles

- Ajouter l'équivalence `<->` au langage (priorité, associativité, mise en FNC).
- Ajouter des listes en compréhension en argument des contraintes globales, par exemple `atLeastOne(cat[i][j] for i in 1..N, j in 1..M where region[i][j] = z)`, et comparer avec votre solution de la question 6.4.
- Écrire dans le langage les modèles d'autres problème et les résoudre. Essayer d'en déduire les limites d'expressivité du langage.
- Ajouter des contraintes globales `count` ou `sum` (encodages à base d'additionneurs ou de compteurs).
- Implémenter l'encodage *par ordre* des domaines finis (variables booléennes $x \leq v$) et le comparer à l'encodage direct.

Références

[Sin05] Carsten Sinz. Towards an optimal CNF encoding of Boolean cardinality constraints. In *Principles and Practice of Constraint Programming (CP 2005)*, volume 3709 of *Lecture Notes in Computer Science*, pages 827–831. Springer, 2005.

[Tse68] Grigori S. Tseitin. On the complexity of derivation in propositional calculus. In A. O. Slisenko, editor, *Studies in Constructive Mathematics and Mathematical Logic, Part II*, pages 115–125. 1968. Traduction anglaise reprise dans *Automation of Reasoning 2*, Springer, 1983, p. 466–483.